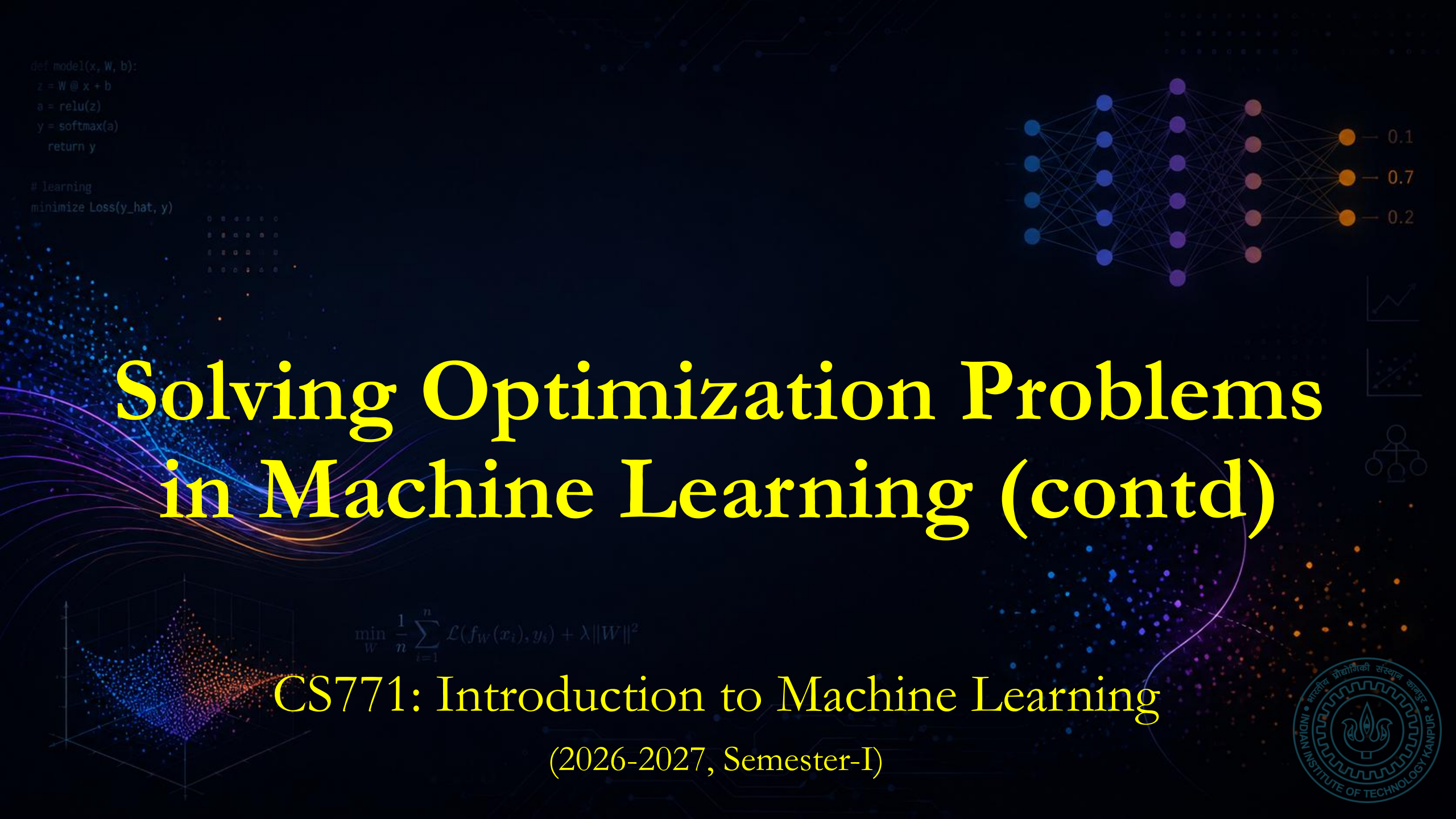


Solving Optimization Problems in Machine Learning (contd)

CS771: Introduction to Machine Learning

(2026-2027, Semester-I)

ML via Optimization: Recap

Most ML problems involve optimizing an objective function of params $\boldsymbol{w}$

$$w_{\text{opt}} = \arg\min_w \mathcal{L}(\boldsymbol{w})$$

$$w_{\text{opt}} = \arg\min_w \mathcal{L}_{reg}(\boldsymbol{w})$$

$$= \arg\min_w [\mathcal{L}(\boldsymbol{w}) + \lambda R(\boldsymbol{w})]$$

Sometimes possible to solve the above using first-order optimality

$$\nabla\mathcal{L}(\boldsymbol{w}) \text{ or } \nabla\mathcal{L}_{reg}(\boldsymbol{w}) = \boldsymbol{0}$$ ..and solve for $\boldsymbol{w}$ (if possible)

Otherwise use iterative optimization, e.g., Gradient descent (GD) as

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta_t \boldsymbol{g}^{(t)}$$

GD variants also exist (to handle non-differentiable objectives, constraints on parameters, stochastic/mini-batch variants of GD)



There is more to Optimization for ML..

Properties of the objective functions in ML problems

- Convex functions
- Strictly convex and strongly convex functions
- Non-convex functions
- Curvature of a function

Optimization using more than just current gradient

- History of previous gradients
- Second order optimization (using "gradient of gradient" or curvature)

Alternating Optimization (ALT-OPT) and Co-ordinate Decent

Another way of solving optimization with constraints: The Lagrangian



Convex Sets

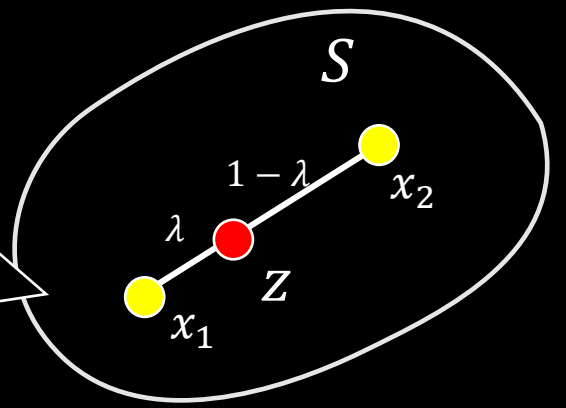
Note: Domain of a convex function is required to be a convex set

For points $x_1, x_2 \in S$, and a $\lambda \in [0,1]$, if point z defined as

A point on the line joining x_1, x_2

$$z = \lambda x_1 + (1-\lambda)x_2$$

.. is in S then set S is called a convex set



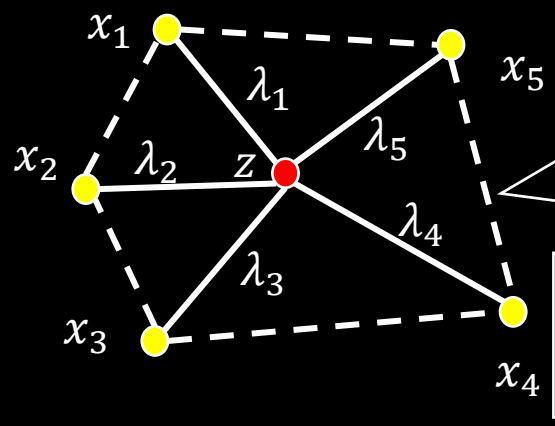
Visual check: For a convex set S, no part of line joining any pair of points $x_1, x_2 \in S$ falls outside S

In the above definition, z is also called a "convex combination" of x_1 and x_2

Side-note: convex combination of N points

$$z = \sum_{i=1}^{N} \lambda_i x_i$$

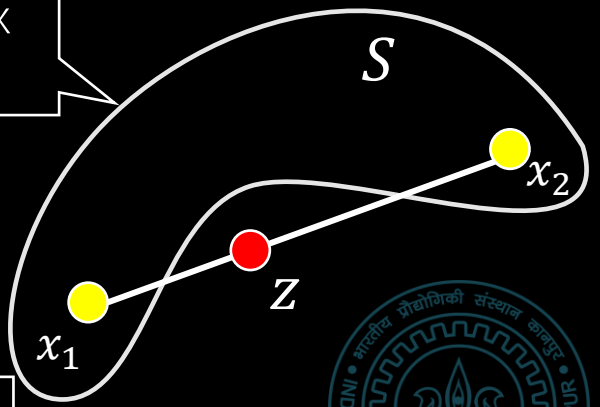
s.t. $\lambda_i \geq 0$ and $\sum_{i=1}^{N} \lambda_i = 1$



Region enclosed by the dashed lines defines the "convex hull" of $x_1, x_2, \ldots, x_5$

The convex hull contains all possible convex combinations of these points

A non-convex set S



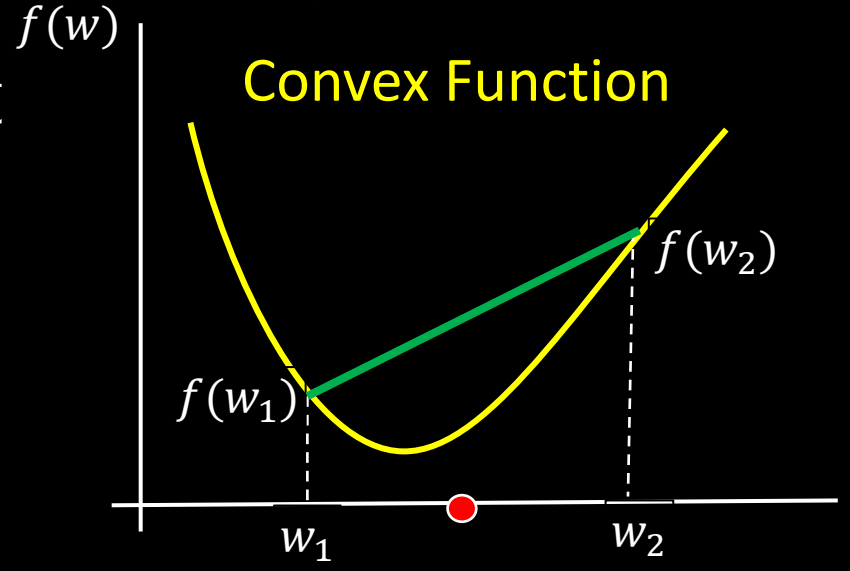
Convex Functions

Jensen's Inequality
($\lambda_i \in [0,1]$ with $\sum_{i=1}^{N} \lambda_i = 1$)

$$f\left(\sum_{i=1}^{n} \lambda_i w_i\right) \leq \sum_{i=1}^{n} \lambda_i f(w_i)$$

If $f(w)$ is convex then all its chords lie above it

$$f(\lambda w_1 + (1-\lambda) w_2) \leq \lambda f(w_1) + (1-\lambda) f(w_2)$$

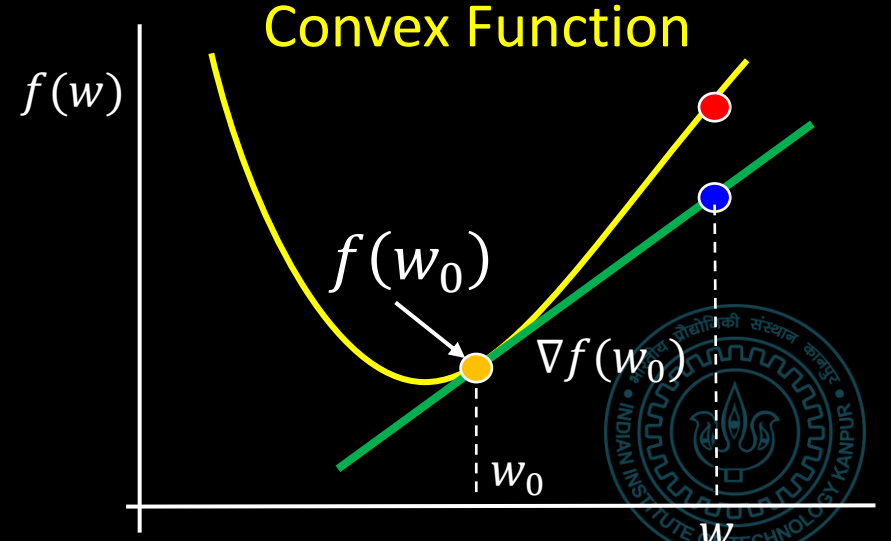


If $f(w)$ is convex + differentiable then it's above its tangents everywhere:

$$f(w) \geq f(w_0) + \nabla f(w_0)^{\mathsf{T}} (w - w_0)$$

Convex functions are "bowl-shaped"

If $f(w)$ is convex then $-f(w)$ is concave (think of concave as an inverted bowl)



Why ML Cares about Convexity?

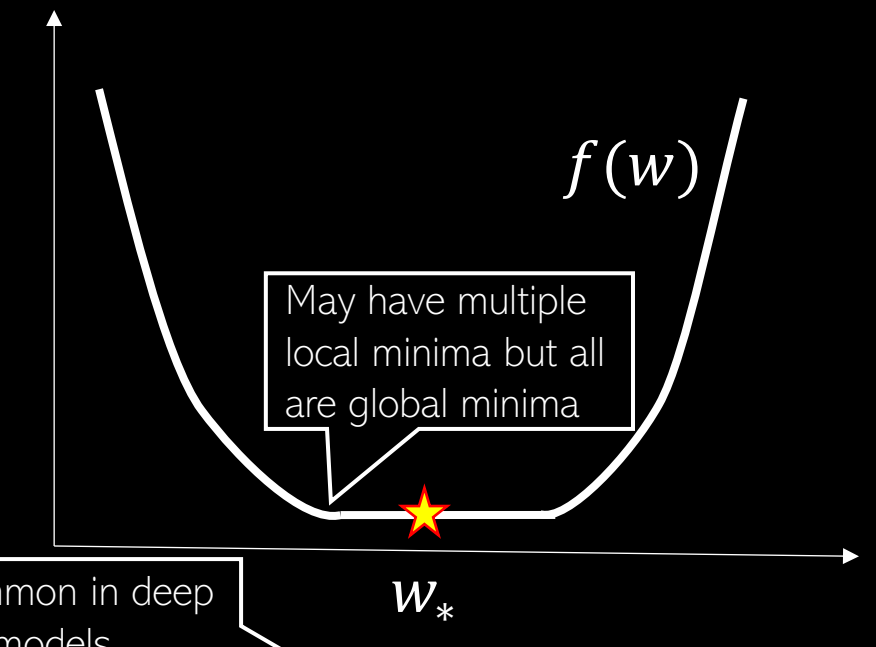
Local minima of a convex $f(w)$ are global minima

If w_* is a minima of a convex $f(w)$ then

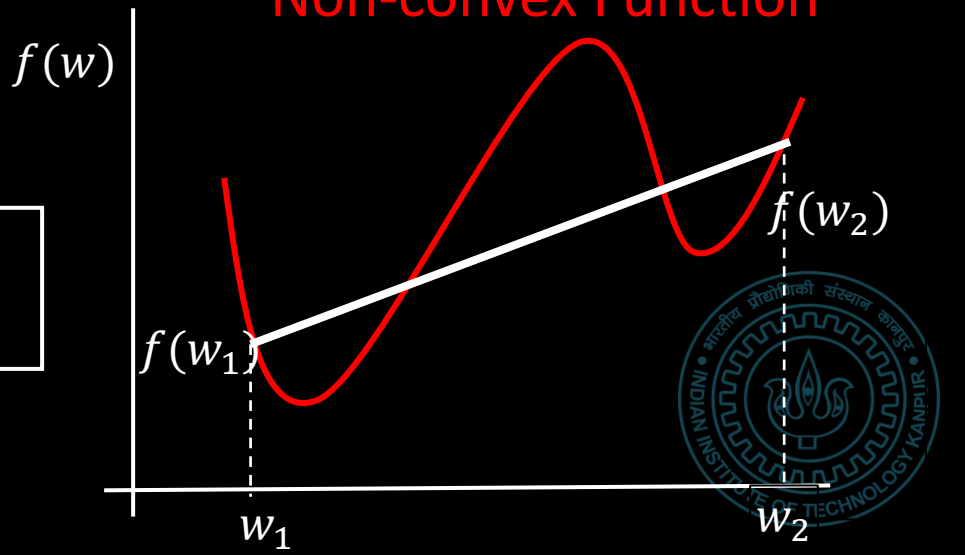
$$f(w_*) \leq f(w) \quad \forall w$$

Convexity removes the problem of "bad" local minima as all of them are also global minima ☺

Functions that are neither convex, not concave are non-convex



Non-convex Function



Multiple local minima (with unequal values)

Thus the best minima can be tricky to find

Hessian and Its Usage

Hessian is essentially the second derivative $f''(\boldsymbol{w})$ when $\boldsymbol{w}$ is a vector

Hessian tells us about the curvature of the function at point $\boldsymbol{w}$

$$H = \nabla^2 \mathcal{L}(\boldsymbol{w}) = \begin{bmatrix} \frac{\partial^2 \mathcal{L}}{\partial w_1^2} & \frac{\partial^2 \mathcal{L}}{\partial w_1 w_2} & \cdots & \frac{\partial^2 \mathcal{L}}{\partial w_1 w_D} \\ \frac{\partial^2 \mathcal{L}}{\partial w_2 w_1} & \frac{\partial^2 \mathcal{L}}{\partial w_2^2} & \cdots & \frac{\partial^2 \mathcal{L}}{\partial w_2 w_D} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 \mathcal{L}}{\partial w_D w_1} & \frac{\partial^2 \mathcal{L}}{\partial w_D w_2} & \cdots & \frac{\partial^2 \mathcal{L}}{\partial w_D^2} \end{bmatrix}$$

For any function, a stationary point $\boldsymbol{w}^*$ ($\nabla \mathcal{L}(\boldsymbol{w}^*) = \mathbf{0}$) is a minima if

$\nabla^2 f(\boldsymbol{w}^*) \succ \mathbf{0}$ — Positive definite (PD)

and maxima if

$\nabla^2 f(\boldsymbol{w}^*) \prec \mathbf{0}$ — Negative definite (ND)

PD: All eigenvalues positive
ND: All eigenvalues negatives

Assume $\mathcal{L}(\boldsymbol{w})$: $\mathbb{R}^D \rightarrow \mathbb{R}$ is convex + twice differentiable, then $\forall\, \boldsymbol{w}$ in its domain

$$\nabla^2 f(\boldsymbol{w}) \succcurlyeq \mathbf{0}$$

Positive semi-definite (PSD) matrix (all eigenvalues non-negative)

Strictly Convex Functions

A convex function can have flat regions, but a strictly convex function has no flat regions

Consequence: Strictly convex function has at most one (i.e., unique) minima

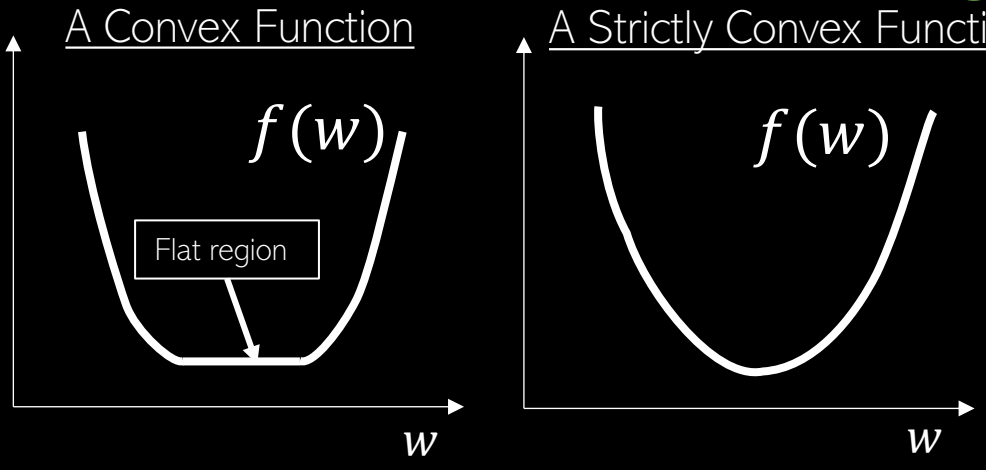
For strictly convex function, all its chords lie strictly above the function

$$f(\lambda w_1 + (1-\lambda)w_2) < \lambda f(w_1) + (1-\lambda) f(w_2)$$

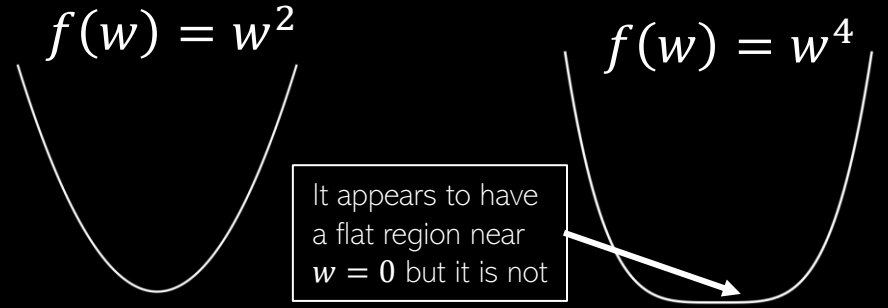
If Hessian is positive-definite $\forall\, \boldsymbol{w}$ then the function must be strictly convex

$$\nabla^2 f(\boldsymbol{w}) \succ \mathbf{0} \;\Rightarrow\; \text{Strictly convex}$$

Note: Reverse is not necessarily true ($f(\boldsymbol{w})$ may be strictly convex but its Hessian may be zero at some points, e.g., $f(w) = w^4$)



Two simple, strictly convex functions



$f''(w) = 2 > 0$

$f''(w) = 12w^2$

Strictly Convex but $f''(0) = 0$

Strongly Convex Functions

Recall that convex functions have non-negative curvature

$$\nabla^2 f(\boldsymbol{w}) \succcurlyeq \mathbf{0}$$

A strongly convex function has an even stronger curvature

A μ-strongly convex function has at least $\mu > 0$ curvature everywhere

$$\nabla^2 f(\boldsymbol{w}) \succcurlyeq \boldsymbol{\mu}$$

Strongly convex ⇒ Strictly Convex ⇒ Convex

Convex

$$f(w) \geq f(w_0) + \nabla f(w_0)^\top (w - w_0)$$

Strictly Convex

$$f(w) > f(w_0) + \nabla f(w_0)^\top (w - w_0)$$

Strongly Convex

$$f(w) \geq f(w_0) + \nabla f(w_0)^\top (w - w_0) + \frac{\mu}{2}\|w - w_0\|^2$$

Extra curvature over standard convex

Strongly convex ≈ Convex with guaranteed minimum curvature

Strong convexity is often assumed to prove faster/stronger convergence guarantees for optimization algorithms such as GD



Convex Functions: Some Common Examples

Such rules exist even when $\boldsymbol{W}$ is a matrix or tensor

Examples: convex $f(z)$ with $z \in \mathbb{R}$

z z^2 $|z|$ $\exp(z)$ $\max(0, z)$

$z^{2k}, k = 1,2, \ldots$ $-\log z, z > 0$

$\log(1 + \exp(z))$

$f(z)$ convex in $z \in \mathbb{R}$ and $z = \boldsymbol{a}^\top \boldsymbol{w} + b$ and terms $\boldsymbol{a} \in \mathbb{R}^D$, $b \in \mathbb{R}$ are constants then $f(\boldsymbol{a}^\top \boldsymbol{w} + \boldsymbol{b})$ is convex in $\boldsymbol{w} \in \mathbb{R}^D$. Popular ML examples:

$z^2 \Leftrightarrow (y - \boldsymbol{w}^\top \boldsymbol{x})^2$ $\quad$ $|z| \Leftrightarrow |y - \boldsymbol{w}^\top \boldsymbol{x}|$

$\max(0, z) \Leftrightarrow \max(0, 1 - y\boldsymbol{w}^\top \boldsymbol{x})$

$\log(1 + \exp(z)) \Leftrightarrow \log(1 + \exp(-y\boldsymbol{w}^\top \boldsymbol{x}))$

Vector norms $\|\boldsymbol{w}\|_p$ $(p \geq 1)$ are convex

Quadratic function

For $\boldsymbol{A} \succcurlyeq 0$, $f(\boldsymbol{w}) = \boldsymbol{w}^\top \boldsymbol{A}\boldsymbol{w} + \boldsymbol{b}^\top \boldsymbol{w} + \boldsymbol{c}$ is convex in $\boldsymbol{w} \in \mathbb{R}^D$

Verify that $\nabla^2 f(\boldsymbol{w}) \succcurlyeq 0$

For any $\boldsymbol{A}, \boldsymbol{b}$, $f(\boldsymbol{w}) = \|\boldsymbol{A}\boldsymbol{w} - \boldsymbol{b}\|^2 = (\boldsymbol{A}\boldsymbol{w} - \boldsymbol{b})^\top (\boldsymbol{A}\boldsymbol{w} - \boldsymbol{b})$ is convex in $\boldsymbol{w} \in \mathbb{R}^D$

Common regularizers such as $\|\boldsymbol{w}\|_1$ and $\|\boldsymbol{w}\|_2^2 = \boldsymbol{w}^\top \boldsymbol{w}$ are convex

$f_1, f_2, \ldots, f_N$ convex $\Rightarrow f \stackrel{\text{def}}{=} \sum_{i=1}^N f_i$ is convex

$f_1, f_2, \ldots, f_N$ convex $\Rightarrow f \stackrel{\text{def}}{=} \max_i \{f_i\}$ is convex



Improving Upon Standard GD



Remembering Past Gradients Helps

The Generic "Remember The Past Gradients" Algorithm

Direction of next update of $\boldsymbol{w}$ incorporates history till now

Using history stabilizes the parameter updates

$$\boldsymbol{g}^{(t)} = \nabla \mathcal{L}(\boldsymbol{w}^{(t)})$$

History/summary of past gradients

$$\boldsymbol{h}^{(t)} = \text{UpdateHistory}(\boldsymbol{h}^{(t-1)}, \boldsymbol{g}^{(t)})$$

$$\tilde{\boldsymbol{g}}^{(t)} = \text{Direction}(\boldsymbol{h}^{(t)}, \boldsymbol{g}^{(t)})$$

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta_t \tilde{\boldsymbol{g}}^{(t)}$$

Different algos differ in how the history is used and updated

For standard GD, there is zero gradient history and $\tilde{\boldsymbol{g}}^{(t)} = \boldsymbol{g}^{(t)}$

Some algos that do use history: Momentum, AdaGrad, RMSProp, Adam

Some Algos That Use Gradient History

Momentum does smoothing in all dimensions of $\boldsymbol{w}$, AdaGrad and RMSProp do scaling dimension-wise

AdaGrad

$$\boldsymbol{h}^{(t)} = \boldsymbol{h}^{(t-1)} + \boldsymbol{g}^{(t)} \odot \boldsymbol{g}^{(t)}$$

$$\tilde{\boldsymbol{g}}_d^{(t)} = \frac{g_d}{\sqrt{\epsilon + h_d^{(t)}}} \quad \forall \; d$$

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta_t \tilde{\boldsymbol{g}}^{(t)}$$

AdaGrad Idea: If past gradients were large in some dimensions ($h_d^{(t)}$ captures that for dimension d), use a reduced effective gradient $\tilde{\boldsymbol{g}}_d^{(t)}$ for updates in those dimensions of $\boldsymbol{w}$

Momentum

$$\boldsymbol{h}^{(t)} = \beta_t \boldsymbol{h}^{(t-1)} + (1 - \beta_t)\boldsymbol{g}^{(t)}$$

$$\tilde{\boldsymbol{g}}^{(t)} = \boldsymbol{h}^{(t)}$$

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta_t \tilde{\boldsymbol{g}}^{(t)}$$

Momentum Idea: Define history as a moving average of the past gradient history and current gradient and use that as effective gradient $\tilde{\boldsymbol{g}}^{(t)}$ (smoother than $\boldsymbol{g}^{(t)}$).

RMSProp: AdaGrad with a moving average

$$\boldsymbol{h}^{(t)} = \beta_t \boldsymbol{h}^{(t-1)} + (1 - \beta_t)\boldsymbol{g}^{(t)} \odot \boldsymbol{g}^{(t)}$$

Adam: Combination of Momentum and RMSProp

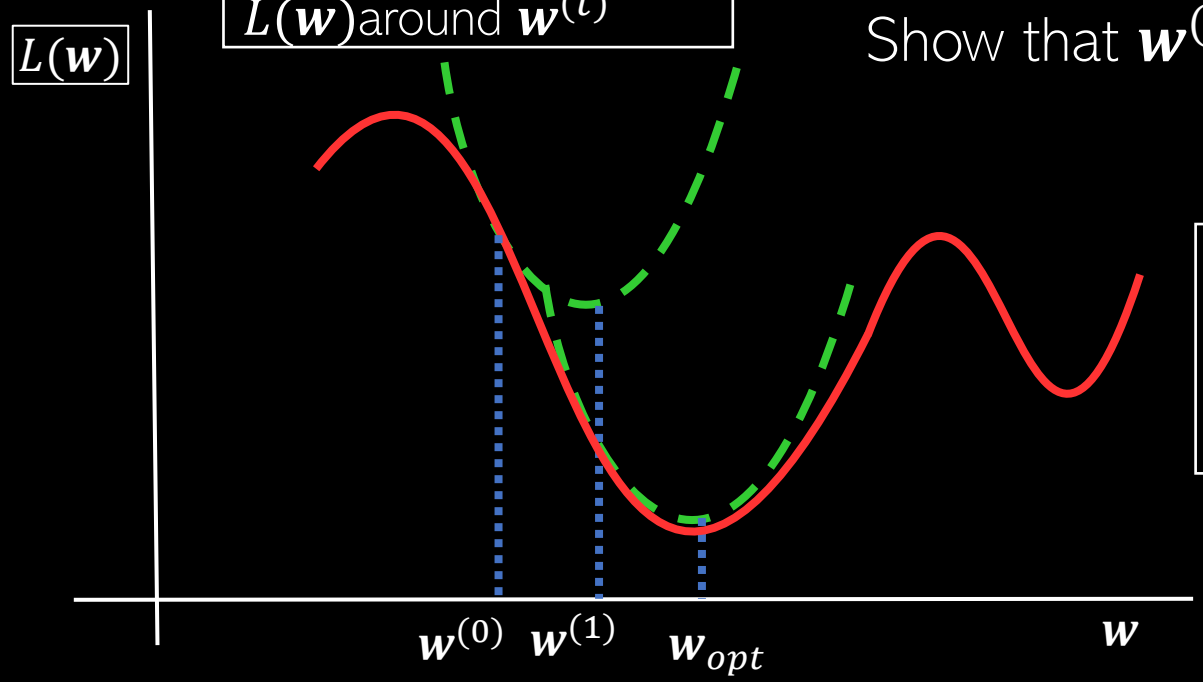
Second-Order Optimization: Newton's Method

- Uses second-order information (the Hessian). Iterative method, just like GD
- Given current $\boldsymbol{w}^{(t)}$, minimize a quadratic approx. of $\boldsymbol{L(w)}$

Quadratic approx

$$\boldsymbol{w}^{(t+1)} = \arg\min_{\boldsymbol{w}} \left[L(\boldsymbol{w}^{(t)}) + \nabla L(\boldsymbol{w}^{(t)})^{\top} (\boldsymbol{w} - \boldsymbol{w}^{(t)}) + \frac{1}{2} (\boldsymbol{w} - \boldsymbol{w}^{(t)})^{\top} \nabla^2 L(\boldsymbol{w}^{(t)}) (\boldsymbol{w} - \boldsymbol{w}^{(t)}) \right]$$

First order approx. of $L(\boldsymbol{w})$ around $\boldsymbol{w}^{(t)}$



Show that $\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \left(\nabla^2 L(\boldsymbol{w}^{(t)})\right)^{-1} \nabla L(\boldsymbol{w}^{(t)})$

$= \boldsymbol{w}^{(t)} - (\boldsymbol{H}^{(t)})^{-1} \boldsymbol{g}^{(t)}$

But per iteration cost is slower due to Hessian computation and inversion

Converges much faster than GD. Also no "learning rate" η_t.



Faster versions exist, e.g., by approximating Hessian using previous gradients (e.g., L-BFGS)

Some Other Useful Optimization Algos



Alternating Optimization (ALT-OPT)

w_A	w_B

Suppose entries in $\boldsymbol{w} \in \mathbb{R}^{\boldsymbol{D}}$ are divided into two non-overlapping groups $\boldsymbol{w}_A$ and $\boldsymbol{w}_B$ s.t. $\dim(\boldsymbol{w}_A) + \dim(\boldsymbol{w}_B) = D$ and the goal is

$$\boldsymbol{w}^{(opt)} = \operatorname{argmin}_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}) = \operatorname{argmin}_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}_A, \boldsymbol{w}_B)$$

Suppose first order optimality doesn't give closed form solution. And GD, though works, is slow if done on the entire $\boldsymbol{w}$ at once

In such cases, often the following "sub-problems" are easy to solve:

$$\boldsymbol{w}_A^{(opt)} = \operatorname{argmin}_{\boldsymbol{w}_A} \mathcal{L}(\boldsymbol{w}_A, \widehat{\boldsymbol{w}}_B)$$

$\boldsymbol{w}_B$ held fixed at $\widehat{\boldsymbol{w}}_B$

$$\boldsymbol{w}_B^{(opt)} = \operatorname{argmin}_{\boldsymbol{w}_B} \mathcal{L}(\widehat{\boldsymbol{w}}_A, \boldsymbol{w}_B)$$

$\boldsymbol{w}_A$ held fixed at $\widehat{\boldsymbol{w}}_A$

Idea: Solve $\boldsymbol{w}^{(opt)} = \operatorname{argmin}_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w})$ by solving these two subproblems in an alternating fashion till we converge (ALT-OPT algo)

ALT-OPT also work for more than 2 groups of variables

Alternating Optimization (ALT-OPT)

Each group itself may be a set of parameters

w_1	w_2	.....	w_G

Assuming $\boldsymbol{w}$ partitioned into G groups $[\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_G]$, and the t-th iteration of ALT-OPT produced estimates $\boldsymbol{w}^{(t)} = [\boldsymbol{w}_1^{(t)}, \boldsymbol{w}_2^{(t)}, \ldots, \boldsymbol{w}_G^{(t)}]$, the next update of each group will be (assuming groups are updated in a cyclical order)

$$\boldsymbol{w}_1^{(t+1)} = \text{argmin}_{\boldsymbol{w}_1} \mathcal{L}(\boldsymbol{w}_1, \boldsymbol{w}_2^{(t)}, \boldsymbol{w}_3^{(t)}, \ldots, \boldsymbol{w}_{G-1}^{(t)}, \boldsymbol{w}_G^{(t)})$$

$$\boldsymbol{w}_2^{(t+1)} = \text{argmin}_{\boldsymbol{w}_2} \mathcal{L}(\boldsymbol{w}_1^{(t+1)}, \boldsymbol{w}_2, \boldsymbol{w}_3^{(t)}, \ldots, \boldsymbol{w}_{G-1}^{(t)}, \boldsymbol{w}_G^{(t)})$$

$$\boldsymbol{w}_3^{(t+1)} = \text{argmin}_{\boldsymbol{w}_3} \mathcal{L}(\boldsymbol{w}_1^{(t+1)}, \boldsymbol{w}_2^{(t+1)}, \boldsymbol{w}_3, \ldots, \boldsymbol{w}_{G-1}^{(t)}, \boldsymbol{w}_G^{(t)})$$

$$\vdots$$

$$\boldsymbol{w}_{G-1}^{(t+1)} = \text{argmin}_{\boldsymbol{w}_{G-1}} \mathcal{L}(\boldsymbol{w}_1^{(t+1)}, \boldsymbol{w}_2^{(t+1)}, \boldsymbol{w}_3^{(t+1)}, \ldots, \boldsymbol{w}_{G-1}, \boldsymbol{w}_G^{(t)})$$

$$\boldsymbol{w}_G^{(t+1)} = \text{argmin}_{\boldsymbol{w}_G} \mathcal{L}(\boldsymbol{w}_1^{(t+1)}, \boldsymbol{w}_2^{(t+1)}, \boldsymbol{w}_3^{(t+1)}, \ldots, \boldsymbol{w}_{G-1}^{(t+1)}, \boldsymbol{w}_G)$$

Each group's update uses the most recently computed values of the other variables



Under suitable conditions (e.g., convex sub-problems), ALT-OPT monotonically decreases the objective and converges to a stationary or groupwise-optimal solution.

Co-ordinate-wise Optimization

In each iteration, update one scalar-valued co-ordinate w_d of $\boldsymbol{w} \in \mathbb{R}^D$

$$w_d^{(t+1)} = \text{argmin}_{w_d} \mathcal{L}(\widehat{w}_1, \ldots, \widehat{w}_{d-1}, w_d, \widehat{w}_{d+1}, \ldots, \widehat{w}_D) \quad \forall \; d$$

.. where $\widehat{w}_i$ is the most recent value of coordinate $i \neq d$

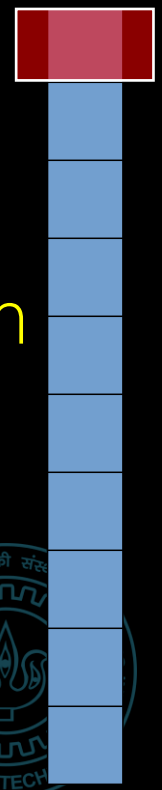
It's a one-dimensional optimization problem (often easy to solve)

If solving above is still not "easy" (e.g., can't solve for w_d in closed form) then we can update w_d via gradient based iterative optimization

Coordinate Descent (CD)

$$w_d^{(t+1)} = w_d^{(t)} - \eta_t g_d^{(t)} \quad \forall \; d$$

.. where $g_d^{(t)} = \nabla_{w_d} \mathcal{L}(\boldsymbol{w})$ is the partial derivative computed at the most recent values of entries in $\boldsymbol{w} = [w_1, w_2, \ldots, w_D]$



Co-ordinate Descent: Some Aspects

- Various possibilities for what happens at each iteration $t+1$ of CD
  - Updating only one randomly picked co-ordinate before going to next iteration
  - Doing a full sweep over all D co-ordinates (*but still updating one at a time*) in a cyclic order or randomly chosen order, and *then* going to next iteration
  - Updating multiple co-ordinates together: block co-ordinate descent (BCD)

We can cache some results (needed for gradient evaluations) for efficient evaluation of gradients (every $\boldsymbol{w}_d$ update needs it) across updates

For example, when only $\boldsymbol{w}_d$ changed between iterations, we can cache the result $s = \boldsymbol{w}^\top \boldsymbol{x}$ (often needed in gradient) and do

$$s^{new} = s^{old} + \left(w_d^{new} - w_d^{old}\right)x_d$$



Lagrangian based Optimization

Consider a constrained optimization problem

$$\boldsymbol{w}^{opt} = \mathbf{argmin}_{\boldsymbol{w}}\, \mathcal{L}(\boldsymbol{w}) \quad \text{s.t.} \quad g(\boldsymbol{w}) \leq \mathbf{0}$$

Let's encode the constraint as follows

$$c(\boldsymbol{w}) = \max_{\alpha \geq 0} \alpha g(\boldsymbol{w}) = \begin{cases} 0, & g(w) \leq 0 \\ \infty, & g(w) > 0 \end{cases}$$

Constraint satisfied

Constraint violated

Thus the constrained problem is equivalently

$$\boldsymbol{w}^{opt} = \mathbf{argmin}_{\boldsymbol{w}}\, \max_{\alpha \geq 0} [\mathcal{L}(\boldsymbol{w}) + \alpha g(\boldsymbol{w})]$$

Minimize subject to worst possible violation of constraints

The Lagrangian: $\mathcal{J}(w, \alpha) = \mathcal{L}(\boldsymbol{w}) + \alpha g(\boldsymbol{w})$

Lagrangian: Primal and Dual Problems

Recall the Lagrangian

$$\mathcal{J}(w, \alpha) = \mathcal{L}(\boldsymbol{w}) + \alpha g(\boldsymbol{w}) \qquad \alpha \geq \mathbf{0}$$

Our original problem (also called the "primal" problem) was

$$p^* = \min_{\boldsymbol{w}} \max_{\alpha \geq 0} \mathcal{J}(w, \alpha)$$

Solve inner max first and then the outer min

The "dual" problem is

$$d^* = \max_{\alpha \geq 0} \min_{\boldsymbol{w}} \mathcal{J}(w, \alpha)$$

Solve inner min first and then the outer max

Weak duality says that this condition always holds: $d^* \leq p^*$

Under suitable convexity/regularity conditions $d^* = p^*$ Strong duality



Lagrangian based Optimization

Consider multiple inequality and equality constraints

$$\boldsymbol{w}^{opt} = \operatorname{argmin}_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w})$$

$$\text{s.t.} \quad g_i(\boldsymbol{w}) \leq 0, \qquad i = 1,2,\dots,K$$

$$h_j(\boldsymbol{w}) = 0, \qquad j = 1,2,\dots,L$$

Side note: In the theory of Lagrange Optimization, there are Karush–Kuhn–Tucker (KKT) conditions that characterize the constrained optima – we will discuss them when we encounter an ML problem where they become relevant

Why is $\alpha_i \geq 0$ and β_j can have any sign?

Introduce one Lagrange multiplier per constant $\alpha_i \geq 0$ and $\beta_j \in \mathbb{R}$

The Lagrangian becomes

$$\mathcal{J}(\boldsymbol{w}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \mathcal{L}(\boldsymbol{w}) + \sum_{i=1}^{K} \alpha_i g_i(\boldsymbol{w}) + \sum_{j=1}^{L} \beta_j h_j(\boldsymbol{w})$$



Lagrangians, Constraints and Regularization

Find $\boldsymbol{w}$ with minimum loss $\mathcal{L}(\boldsymbol{w})$ with complexity $R(\boldsymbol{w})$ below $C > 0$

$$\min_w \mathcal{L}(\boldsymbol{w}) \text{ s.t. } R(\boldsymbol{w}) \leq C$$

The Lagrangian:

$$\min_w [\mathcal{L}(\boldsymbol{w}) + \alpha(R(\boldsymbol{w}) - C)]$$

$$\Rightarrow \min_w [\mathcal{L}(\boldsymbol{w}) + \alpha R(\boldsymbol{w})]$$

Find $\boldsymbol{w}$ with minimum complexity $R(\boldsymbol{w})$ with loss $\mathcal{L}(\boldsymbol{w})$ below $\epsilon > 0$

$$\min_w R(\boldsymbol{w}) \text{ s.t. } \mathcal{L}(\boldsymbol{w}) \leq \epsilon$$

The Lagrangian:

$$\min_w [R(\boldsymbol{w}) + \alpha(\mathcal{L}(\boldsymbol{w}) - \epsilon)]$$

$$\Rightarrow \min_w [\mathcal{L}(\boldsymbol{w}) + \frac{1}{\alpha} R(\boldsymbol{w})]$$

Scaling by a constant α doesn't change the minima

Thus optimization with constraints is closely related to adding a regularization term to the loss function $\min_w [\mathcal{L}(\boldsymbol{w}) + \lambda R(\boldsymbol{w})]$



GD Performs Implicit Regularization in Over-parameterized Models



Optimization for Over-parameterized Models

Consider linear regression with N training examples and P features

$$\boldsymbol{\Phi w} = \boldsymbol{y} \qquad \boldsymbol{\Phi} \in \mathbb{R}^{N \times P}, \boldsymbol{w} \in \mathbb{R}^{P}, \boldsymbol{y} \in \mathbb{R}^{N}$$

If $P > N$, the model is called over-parametrized.

Here, infinite many solutions for $\boldsymbol{w}$ exist that achieve zero training error

In such cases, we prefer $\boldsymbol{w}$ with smallest norm. This amounts to solving

$$\boldsymbol{w}_* = \arg\min_{\boldsymbol{w}} \|\boldsymbol{w}\|_2^2 \qquad \text{s.t.} \quad \boldsymbol{\Phi w} = \boldsymbol{y}$$

Note that the above is different from solving ridge regression where we do

$$\boldsymbol{w}_* = \arg\min_{\boldsymbol{w}} \left\{ \frac{1}{N} \|\boldsymbol{y} - \boldsymbol{\Phi w}\| + \lambda \|\boldsymbol{w}\|_2^2 \right\}$$

Zero training error is usually not possible unlike overparametrized case (also, $N > D$)

Not looking for the smallest norm $\boldsymbol{w}$ but just some $\boldsymbol{w}$ that minimizes the loss + regularizer

In O.P. Linear Regression, GD Finds Min-Norm $\boldsymbol{w}$

We can explicitly try to solve the minimum norm problem

$$\boldsymbol{w}_* = \arg\min_{\boldsymbol{w}} \|\boldsymbol{w}\|_2^2 \qquad \text{s.t.} \quad \boldsymbol{\Phi w} = \boldsymbol{y}$$

Without asking GD to also ensure that $\boldsymbol{w}$ has the smallest norm

But if we just use gradient descent (GD) to optimize the following w.r.t. $\boldsymbol{w}$

$$\mathcal{L}(\boldsymbol{w}) = \|\boldsymbol{y} - \boldsymbol{\Phi w}\|^2 = \sum_{n=1}^{N} (y_n - \boldsymbol{w}^\top \phi(\boldsymbol{x}_n))^2$$

.. and initialize GD with $\boldsymbol{w}^{(0)} = \boldsymbol{0}$, the GD solution will be

$$\boldsymbol{w}_{GD} = \sum_{n=1}^{N} \alpha_n \, \phi(\boldsymbol{x}_n)$$

Note that this has the same form as the Representer Theorem based solution of regularized loss functions

Recall the GD updates are of the form $\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} + \eta \sum_{n=1}^{N} \text{error}_n \times \phi(\boldsymbol{x}_n)$

.. which is also min. norm solution because $\boldsymbol{w}_\perp = \boldsymbol{0}$

Using a similar argument as in Representer Theorem case.

Thus GD acts as an "implicit regularizer" in over-param cases